SecFlowOps: Auditable DevSecOps Remediation and Policy-Gate Evaluation

Stéphane Gaël R. Ekodeck, Serge Alain Ebele, and Abdoul Wabou Elh Mal Yaya

***Abstract*—DevSecOps evaluations frequently emphasize scanner counts, even though release decisions depend on whether findings survive remediation and violate explicit policy gates. SecFlowOps addresses this gap with an executable evaluation framework that records the trajectory from scanner finding to remediation, rescan, residual-risk measurement, and OPA policy decision. It integrates Trivy, Semgrep, Gitleaks, npm audit, OWASP ZAP, and OPA without treating any tool as proof of system security. The evaluation covers a 432-run controlled protocol, a 90-run controlled size study, a 75-run open-source campaign, a NodeGoat npm-remediation study, an injected external recall study, and a targeted ZAP DAST probe. SecFlowOps reaches recall 1.0 on injected external ground truth and removes all injected target findings after remediation. In the 432-run protocol, mean residual findings decrease from 28.0 under scan-only configurations to 16.0 under remediation configurations. On NodeGoat, npm-only remediation reduces findings from 78 to 32, while SecFlowOps reduces them to 31 by also removing a committed secret; residual critical findings remain blocked by policy. The contribution is an auditable framework for comparing scanner-to-policy trajectories, not evidence that arbitrary software can be automatically repaired or safely deployed.**

***Index Terms*—DevSecOps, remediation, policy-as-code, benchmarking, reproducibility, security.**

I. Introduction

DEVSECOPS pipelines often combine scanners, dependency updates, local patches, and deployment gates, but empirical claims about these combinations are difficult to audit. A scanner-only pipeline can find issues without changing delivery behavior. A remediation pipeline can edit code without proving that the residual state satisfies an explicit policy. A policy gate can block delivery without showing whether remediation actually reduced risk.

The information-security object studied here is the release-governing evidence produced by a pipeline: scanner observations, remediation traces, residual-risk measurements, and policy decisions. This framing treats DevSecOps automation as a measurable security system rather than as a software-engineering convenience alone.

SecFlowOps is designed around this missing unit of analysis: the trajectory of a security finding from initial detection, through remediation and rescan, to a policy decision over the residual state. The contribution is not a new vulnerability scanner and not an autonomous repair system. It is a reproducible framework for evaluating DevSecOps pipeline configurations under common scanner, remediation, and gate workloads, including cases where the appropriate outcome is policy denial rather than delivery.

Table I gives the research questions, their literature motivation, and the evidence used to answer them. The questions follow from prior work showing that static-analysis warnings require workflow integration [1], [2], security-tool benchmarks need explicit measurement criteria [3], and dependency remediation must be evaluated beyond the existence of automated update mechanisms [4], [5].

The main contributions are as follows:

1) a finding-trajectory measurement model that records initial findings, residual findings, ground-truth matches where complete ground truth exists, remediation traces, and OPA delivery decisions;
2) an executable DevSecOps evaluation framework that integrates Trivy, Semgrep, Gitleaks, npm audit, OWASP ZAP, and OPA without treating any single tool as proof of system security;
3) an empirical evaluation over controlled, open-source, NodeGoat, injected-external, and DAST-focused campaigns, with security outcomes and performance costs reported separately;
4) reproducibility evidence covering executable procedures, scanner observations, processed metrics, integrity checks, tool status, and remote CI and pull-request validation.

II. Background and Related Work

Secure-development and DevSecOps guidance emphasizes continuous vulnerability management, supply-chain controls, feedback loops, and measurable assurance activities [6]–[8]. The multivocal DevSecOps literature also frames security as an integrated development and operations concern rather than a final audit step [9]. These sources motivate pipeline automation, but they do not define a concrete experimental protocol for comparing scanner-only, remediation, and policy-gated configurations under identical workloads.

Prior empirical work shows why scanner counts alone are insufficient. Developers may avoid static analysis tools when warnings are hard to prioritize or integrate into workflow [1]. Large-scale studies of static analysis in open source also show that tool adoption, warning interpretation, and project context affect the practical value of analysis results [2]. Benchmarking work for vulnerability testing tools further argues that evaluation should specify targets, measurements, and comparability criteria rather than relying on informal demonstrations [3].

Stéphane Gaël R. Ekodeck, Serge Alain Ebele, and Abdoul Wabou Elh Mal Yaya are with the University of Yaounde I, LIRIMA, TEAM GRIMCAPE, Yaoundé, Cameroon.

Stéphane Gaël R. Ekodeck is also affiliated with IRD, UMI, UMMISCO, IRD France Nord, F-93143 Bondy, France, and Sorbonne Universités, Univ. Paris 06, UMI 209, UMMISCO, F-75005 Paris, France. Corresponding author: stephane-gael.ekodeck@facsciences-uy1.cm.

TABLE I
RESEARCH QUESTIONS, MOTIVATION, AND SUPPORTING EVIDENCE.

RQ	Question	Motivation	Evidence used to answer
RQ1	Does SecFlowOps detect and remove injected ground-truth findings in controlled and real-codebase contexts?	Scanner outputs alone do not establish whether known target findings are detected and removed [1], [2].	Controlled and injected-external recall, residual ground-truth counts, ZAP probe.
RQ2	How do scanner-only, remediation-only, policy-only, and combined configurations differ in residual findings and policy outcomes?	Benchmark comparisons require fixed targets, comparable configurations, and explicit outcome criteria [3].	Configuration table, 432-run protocol, external OSS campaign, NodeGoat baseline.
RQ3	What runtime cost is introduced by remediation, rescanning, npm updates, DAST, and OPA policy evaluation?	Automated dependency updates and pipeline gates are useful only if operational cost is measured with the security outcome [4], [5].	Component timings, paired overheads, full-protocol and NodeGoat performance summaries.

Dependency security and automated updates raise a separate measurement problem. Vulnerabilities in npm dependency networks can propagate through package graphs [4], and automated pull requests can encourage dependency upgrades but still require evaluation of acceptance, failures, and residual risk [5]. SecFlowOps therefore treats package-manager remediation as one measurable component of a pipeline rather than as a complete security solution.

SecFlowOps builds on existing tools rather than replacing them. Trivy provides vulnerability, secret, and misconfiguration scanning over source and dependency ecosystems [10]. Semgrep supplies rule-driven SAST for source code [11]. Gitleaks detects committed secrets [12]. npm audit supplies advisory-driven JavaScript dependency remediation [13]. OWASP ZAP supplies dynamic web-application scanning [14]. OPA provides policy-as-code enforcement through Rego and structured input decisions [15]. The original contribution is the integration and measurement layer: normalized finding records, explicit ground-truth linkage where available, deterministic and package-manager remediation traces, post-remediation rescans, and policy decisions over residual findings.

Table II states the gaps that drive the design. The comparison is not a head-to-head performance ranking of prior systems; it identifies whether the cited literature category makes the same measurement object available.

III. METHOD

Figure 1 summarizes the SecFlowOps architecture. The framework evaluates an isolated codebase copy rather than editing the original codebase in place. This design makes the unit of measurement a pipeline run, not a permanent project migration.

The pipeline has five stages. First, build or test commands are executed in an isolated codebase copy. For heterogeneous external codebases, the build step can be explicitly skipped so that missing project-specific dependencies do not confound security-pipeline measurement. Second, scanners collect SAST, SCA, secret, Docker, and Kubernetes evidence. Third, findings are normalized into records with stable fingerprints and optional ground-truth links. Fourth, remediation rules patch only the isolated copy. Fifth, a second scanner pass measures residual findings before OPA evaluates the delivery policy.

For each run r, SecFlowOps records an initial finding multiset $F_0(r)$ and a residual finding multiset $F_1(r)$ after remediation. Where non-empty complete injected ground truth $G(r)$ is available, recall is computed as $|\mathrm{match}(F_0(r), G(r))|/|G(r)|$, and residual ground-truth findings are computed as $|\mathrm{match}(F_1(r), G(r))|$. These quantities are not reported as natural-vulnerability recall for open-source codebases because no complete independent ground truth exists for those codebases. The policy decision is computed only from $F_1(r)$ and the configured thresholds, so a run may be successful as an experiment while correctly denying delivery.

Remediation is deliberately bounded. Deterministic rules update known vulnerable Python pins, remove documented fake secrets, escape reflected HTML output, parameterize a SQL query, and harden Docker or Kubernetes settings. For NodeGoat, the npm remediator runs `npm audit fix --package-lock-only --omit=dev` and records the npm return code and dependency-manifest change. A non-zero npm return code is retained as evidence of residual advisories, not hidden as a tool failure.

OPA evaluates residual findings using a policy with zero tolerance for critical findings, a high-severity threshold of three, a CVSS ceiling of 9.0, and blocking on residual secrets. The policy counts residual findings as a list rather than a set so duplicate scanner findings are not silently deduplicated.

IV. EXPERIMENTAL DESIGN

Table III defines the configurations used in the experiments. The notation is used consistently in the results tables and figures.

Table IV summarizes the campaigns. The controlled campaign evaluates six configurations: build only, non-blocking scanning, automatic scanning, policy only, agents only, and the combined SecFlowOps configuration. Five generated codebases inject SAST, SCA, secret, Docker, and Kubernetes findings with explicit ground-truth labels. Each controlled codebase is evaluated three times, yielding 90 runs.

The full protocol campaign executes the complete local design: three codebase families, eight scenarios, six configurations, and three repetitions, yielding 432 runs. The eight scenarios are clean, vulnerable dependency, fake secret, reflected XSS, SQL injection, Docker misconfiguration, Kubernetes misconfiguration, and multi-layer. The 432-run protocol records the DAST scenario as ground truth, while executable

TABLE II
POSITIONING AGAINST CITED LITERATURE CATEGORIES. "MEASURED" MEANS THAT THE SOURCE CATEGORY EXPLICITLY EVALUATES THE CRITERION AS A REPORTED OUTCOME.

Criterion	Secure-development guidance [6]–[8]	Static-analysis and benchmark studies [1]–[3]	Dependency-remediation studies [4], [5]	SecFlowOps
Finding-to-policy trajectory	Recommended conceptually	Not the main measured object	Not the main measured object	Measured per run
Post-remediation rescan	Not operationalized	Partially addressed through tool evaluation	Partially addressed for dependencies	Measured per run
Explicit policy-gate decision	Recommended conceptually	Not measured	Not measured as residual gate outcome	Measured with OPA
Injected ground-truth recall	Not applicable	Benchmark-dependent	Not applicable	Measured where ground truth exists
External OSS residual findings	Not measured	Tool/output focused	Dependency focused	Measured with adjudication caveats
Runtime overhead by component	Not measured	Sometimes measured for tools	Not central to residual policy	Measured for scan, remediation, rescan, policy
Reproducible evaluation evidence	Not applicable	Recommended by benchmark logic	Not uniformly provided	Executable procedures, metrics, integrity checks, CI evidence

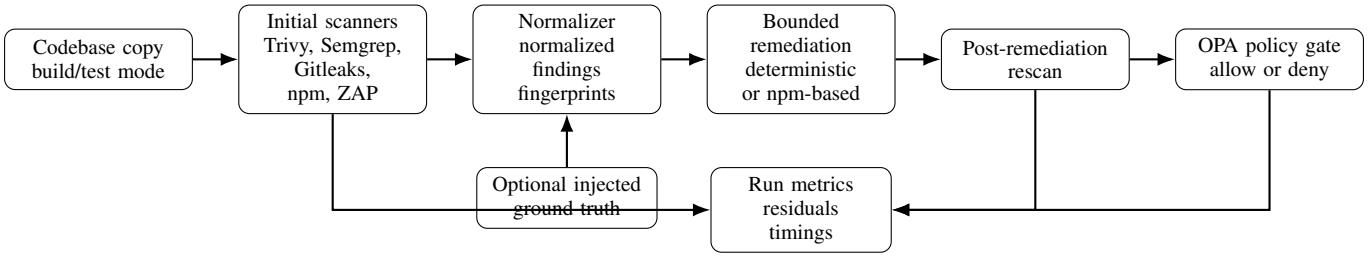

Fig. 1. SecFlowOps workflow and measurement architecture. Scanners produce normalized findings, bounded remediators edit only the isolated copy, a second scanner pass measures residual findings, and OPA decides whether the residual state satisfies the delivery policy.

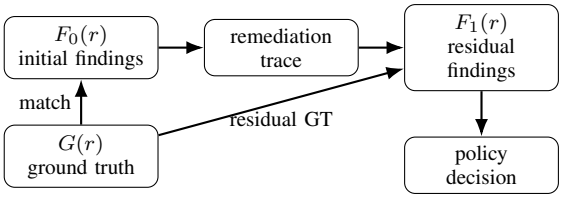

Fig. 2. Finding-trajectory model. Ground-truth metrics are reported only when complete injected ground truth is available.

ZAP coverage is measured in a separate targeted DAST probe to avoid merging a focused runtime validation with the full local protocol.

The external campaign evaluates five configurations, non-blocking scanning, automatic scanning, policy only, agents only, and the combined SecFlowOps configuration, on five open-source codebases: Requests, Flask, Express, OWASP NodeGoat, and DVNA. Each external codebase is evaluated three times, yielding 75 runs. The external campaign uses an explicit build-skip mode to avoid confounding security-pipeline behavior with project-specific dependency installation and test setup.

Two focused campaigns address external validity gaps. A NodeGoat npm study repeats all five configurations three times on OWASP NodeGoat and enables `npm audit fix --package-lock-only --omit=dev` in the isolated remediation copy. A separate npm-audit-only baseline repeats the same package-manager remediation three times without SecFlowOps secret remediation or OPA gating. An injected external recall study copies Requests, Flask, and Express, injects six controlled findings, and evaluates three configurations over three repetitions, yielding 27 runs with complete injected ground truth.

TABLE III
PIPELINE CONFIGURATIONS.

ID	Definition
C0	Build or test only; no security scanner, remediator, rescan, or policy gate.
C1	Non-blocking scanner execution; findings are measured but delivery is not blocked.
C2	Scanner execution in automatic mode without remediation or policy decision.
C3	Policy-only gate over raw findings; no remediation before the decision.
C4	Remediation agents and post-remediation rescan; no OPA gate.
C5	Combined SecFlowOps: scan, remediate, rescan, and OPA policy gate.
B1	npm-audit-only NodeGoat baseline without SecFlowOps secret remediation or OPA gate.

V. SECURITY RESULTS

Table V summarizes the main security outcomes across the controlled, external, NodeGoat, and injected-external studies.

TABLE IV
EVALUATION CAMPAIGNS AND MAIN PURPOSE.

Campaign	Corpus	Runs	Reps	Purpose
Controlled	five generated API variants	90	3	End-to-end behavior with injected ground truth and controlled size variation.
Full protocol	24 generated scenario codebases	432	3	Complete local design: three families, eight scenarios, six configurations, and three repetitions.
External OSS	Requests, Flask, Express, NodeGoat, DVNA	75	3	Scanner, remediation, adjudication, policy, and performance behavior on real codebases.
NodeGoat npm	OWASP NodeGoat	15+3	3	Scanner-only, npm-audit-only, agents-only, and SecFlowOps behavior on a difficult npm dependency graph.
Injected external	Requests, Flask, Express copies with injected findings	27	3	Recall and residual target findings where complete injected ground truth exists in real-codebase contexts.
ZAP DAST probe	reflected-XSS generated codebase	2	1	Validate OWASP ZAP execution, JSON normalization, and DAST ground-truth matching.

TABLE V
MAIN SECURITY OUTCOMES. COUNTS ARE MEANS WHEN SEVERAL CODEBASES OR REPETITIONS ARE AGGREGATED.

Study and configuration	Runs	Success	Initial	Residual	Main interpretation
Controlled C1	15	15	64.0	64.0	Scanner-only baseline; injected-ground-truth recall was 1.0.
Controlled C5	15	15	64.0	16.0	Critical findings and secrets were reduced to zero; two high Kubernetes hardening findings remained per run.
External C1	15	15	18.6	18.6	Natural OSS scan without remediation.
External C5	15	12	18.6	15.4	Flask and DVNA findings were remediated; NodeGoat was denied by policy.
NodeGoat C5	3	0	78.0	31.0	npm remediation and secret removal were partial; policy denial remained correct.
Injected external C5	9	9	68.3	16.0	Injected-ground-truth recall was 1.0 and residual injected findings were zero.

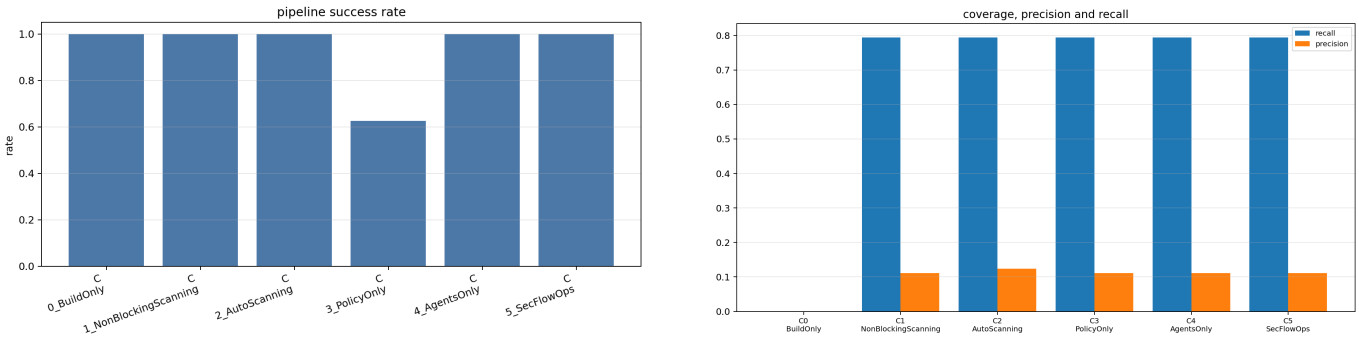

Fig. 3. Full-protocol result figures generated from the processed metrics: pipeline success by configuration and coverage/precision/recall summaries.

In the controlled campaign, five generated codebases were evaluated across six configurations and three repetitions, for 90 runs. The policy-only configuration was blocked in all 15 runs, while the combined SecFlowOps configuration passed in all 15 runs after local remediation and OPA evaluation. For SecFlowOps runs, residual critical findings and residual secrets were reduced to zero. Two high-severity Kubernetes hardening findings remained in each run and were permitted by the configured policy threshold.

The full protocol campaign executed 432/432 planned local runs. Each configuration has 72 runs. Pipeline success was 72/72 for C0, C1, C2, C4, and C5, and 45/72 for C3 because policy-only correctly denied vulnerable raw findings. Mean residual findings decreased from 28.0 in scan-only configurations to 16.0 in C4 and C5. Mean recall was 0.795 in the 432-run campaign because DAST ground-truth rows are part of the scenario design but executable ZAP scanning is evaluated separately. The targeted ZAP probe produced real ZAP JSON output and matched the reflected-XSS DAST ground truth with severity high.

In the external campaign, 75 runs were executed with no tool failures. Non-blocking scanning, automatic scanning, and

TABLE VI
NODEGOAT NPM REMEDIATION AND POLICY OUTCOME.

Configuration	Total	Crit.	High	Secrets	Policy
C1 scanner only	78	10	40	1	N/A
B1 npm audit only	32	3	13	1	N/A
C4 agents only	31	3	12	0	N/A
C5 SecFlowOps	31	3	12	0	Deny

TABLE VII
INJECTED-EXTERNAL RECALL AND REMEDIATION.

Configuration	Runs	Recall	Resid. GT	Outcome
C1 scanner only	9	1.0	N/A	Detection only
C4 agents only	9	1.0	0	Injected findings remediated
C5 SecFlowOps	9	1.0	0	Remediated and policy-passed

agents-only execution passed in 15/15 runs. Policy-only and SecFlowOps each passed in 12/15 runs; the failed runs were policy denials on OWASP NodeGoat. The external campaign produced 82 unique findings and a first-pass static adjudication table. Under this adjudication, SecFlowOps resolved 13/13 Flask example dependency findings, 2/2 DVNA Dockerfile hardening findings, and the committed NodeGoat private key.

Table VI isolates the hardest external codebase. The npm-only baseline shows that `npm audit fix --package-lock-only --omit=dev` resolves many NodeGoat dependency advisories but leaves the committed secret untouched. SecFlowOps reaches a slightly lower residual count because it also removes the committed private key. Residual findings still include 3 critical and 12 high findings, so SecFlowOps remained blocked by policy in all three combined runs.

The injected external recall study completed 27/27 runs successfully. Table VII distinguishes detection from remediation. Recall against injected ground truth was 1.0 for all evaluated configurations. For SecFlowOps, all nine runs ended with zero residual critical findings, zero residual secrets, and zero residual injected ground-truth findings.

A. Precision Interpretation

We report three distinct notions that should not be merged. First, controlled and injected-external recall are measured against known injected ground truth. Second, the conservative precision metric treats all non-ground-truth scanner findings as unknown or false-positive candidates; for the injected external study this yields a mean precision of 0.088 because real-codebase hardening and dependency findings coexist with the injected findings. This number is therefore a lower-bound bookkeeping measure, not a claim that contemporary scanners are mostly wrong. Third, the external adjudication reviewed 82 unique naturally occurring findings by static codebase evidence; within that reviewed subset, adjudicated precision was 1.0 for runs with adjudicated findings. This is not an independent exploitability audit and is not used to claim complete recall over naturally occurring external vulnerabilities.

B. Answers to the Research Questions

Table VIII links the reported evidence back to the research questions in Table I. The answers are intentionally bounded to the campaigns that were executed.

VI. PERFORMANCE RESULTS

The controlled campaign includes a performance study using the same 90 runs. Table IX summarizes the main end-to-end timings. Mean pipeline time was 0.246 seconds for build-only execution, 9.181 seconds for non-blocking scanning, 9.345 seconds for policy-only execution, 17.759 seconds for agents-only execution, and 18.122 seconds for the full SecFlowOps configuration.

For SecFlowOps, the mean component times were 8.819 seconds for the initial scan, 8.553 seconds for the post-remediation scan, 0.288 seconds for local remediation, and 0.121 seconds for the OPA policy gate. Thus, scanner execution dominates runtime; policy evaluation is not the primary performance bottleneck in this controlled setting.

The paired mean overhead of SecFlowOps over non-blocking scanning was 8.941 seconds. This overhead is mainly attributable to remediation validation and the second scanner pass.

In the external campaign, mean pipeline time was 14.671 seconds for non-blocking scanning, 14.133 seconds for automatic scanning, 14.926 seconds for policy-only execution, 27.924 seconds for agents-only execution, and 28.637 seconds for SecFlowOps. For SecFlowOps, mean component times were 14.304 seconds for the initial scanner pass, 13.663 seconds for the post-remediation scanner pass, 0.401 seconds for remediation, and 0.097 seconds for the OPA policy gate. The paired mean overhead of SecFlowOps over non-blocking scanning was 13.966 seconds, again dominated by the second scanner pass.

Table X compares the NodeGoat npm-only baseline with SecFlowOps. The npm-only baseline took 104.518 seconds on average, including 51.078 seconds in `npm audit fix`. SecFlowOps took 96.302 seconds on average, including 44.778 seconds of remediation. These values should be interpreted as reported local timings rather than general npm performance constants because package-manager runtime depends on cache state, dependency-resolution state, and advisory-database state.

In the injected external recall study, mean pipeline time was 18.155 seconds for non-blocking scanning and 35.519 seconds for SecFlowOps. These results show that npm remediation is substantially more expensive than deterministic text patches, and that scanner re-execution remains the dominant overhead across all studies.

In the 432-run full protocol campaign, mean pipeline time was 22.061 seconds for C1 and 43.749 seconds for C5; medians were 20.691 seconds and 41.409 seconds, respectively. One C2 Semgrep run lasted 3944.596 seconds and is retained in the raw data; this outlier explains why C2 mean runtime is much larger than its median. In the ZAP probe, the ZAP-enabled scanner pass took 25.845 seconds for C1 and 20.116 seconds for C3, including ZAP quick scan execution against a local dynamic-port target.

TABLE VIII
EVIDENCE-BASED ANSWERS TO THE RESEARCH QUESTIONS.

RQ	Evidence	Answer
RQ1	Injected-external runs reached recall 1.0 and C5 ended with zero residual injected findings; the targeted ZAP probe produced ZAP JSON output and matched reflected-XSS ground truth.	Supported for injected findings and the ZAP probe. Not claimed for complete natural-vulnerability recall on external codebases.
RQ2	In the 432-run protocol, mean residual findings decreased from 28.0 in scan-only configurations to 16.0 in C4/C5. NodeGoat remained denied because residual critical and high findings survived npm remediation.	Combined scan-remediate-rescan-policy evaluation distinguishes reduction from acceptable delivery; policy denial is a measured outcome, not a tool failure.
RQ3	Full-protocol C5 mean runtime was 43.749 s versus 22.061 s for C1; NodeGoat C5 mean runtime was 96.302 s versus 19.620 s for C1. OPA evaluation was below scanner and remediation costs in the controlled and external campaigns.	The dominant costs are rescanning and npm remediation, not policy evaluation, in the reported environment.

TABLE IX
MEAN END-TO-END RUNTIME BY STUDY AND CONFIGURATION.

Study	C1 scan	C5 SecFlowOps	Overhead
Controlled	9.181 s	18.122 s	8.941 s
External OSS	14.671 s	28.637 s	13.966 s
Injected external	18.155 s	35.519 s	17.364 s
NodeGoat npm	19.620 s	96.302 s	76.682 s
Full protocol	22.061 s	43.749 s	21.688 s

TABLE X
NODEGOAT NPM PERFORMANCE BASELINE.

Configuration	Pipeline	npm/remediation
B1 npm audit only	104.518 s	51.078 s
C5 SecFlowOps	96.302 s	44.778 s

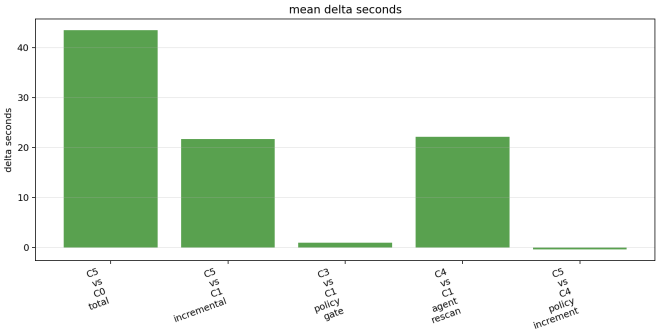

Fig. 4. Full-protocol performance overheads generated from the processed timing metrics.

VII. REPRODUCIBILITY AND ARTIFACT SCOPE

The review repository is available at https://github.com/EkodeckStephane/SecFlowOps-DevSecOps8Abdoul. The artifact is organized to expose the procedures, code, data, and metric-generation evidence needed to recreate the manuscript's tables and figures.

The artifact includes executable scripts for the reported studies: `run_expanded_study.ps1`, `run_full_protocol_study.ps1`, `run_external_study.ps1`, `run_nodegoat_npm_study.ps1`, and `run_injected_external_study.ps1`. It also includes a separate npm-only baseline script, `run_npm_audit_only_baseline.ps1`. The scripts generate raw logs, normalized findings, processed run metrics, summary tables, and performance tables under `SecFlowOps/data`, `SecFlowOps/tables`, and `SecFlowOps/figures`.

The checked local toolchain uses Python 3.13, Docker, Java 24.0.1, Trivy 0.71.2, Semgrep 1.169.0, OPA 1.18.2, Gitleaks 8.30.1, and OWASP ZAP 2.17.0. OPA, Gitleaks, and ZAP are bundled in the artifact, while Trivy, Semgrep, Docker, npm, and Java are expected from the host environment. Tool status is recorded in `SecFlowOps/artifact/tool_status.json`; file checksums are recorded in `SecFlowOps/artifact/checksums.sha256`.

Remote workflow evidence is recorded in `github_actions_evidence.md`. The GitHub validation used a private repository, executed the remote CI workflow on `main`, created a real remediation pull request from a user-pushed branch, observed passing `pull_request` CI, merged the PR, measured a 25 min 35 s creation-to-merge latency, and observed passing post-merge CI on `main`.

Reproducibility is claimed at the artifact-workflow level: a reviewer can rerun the scripts to regenerate metrics using the same source state and installed tools. We do not claim bit-identical future vulnerability counts because Trivy, npm, and advisory databases are time-sensitive. We also do not claim independent replication: no third-party rerun has been performed as part of this manuscript.

VIII. THREATS TO VALIDITY

The controlled campaign uses generated codebases and deterministic patches. The ground truth is injected by construction and does not represent natural project diversity.

The external campaign improves external validity by using five real open-source codebases and a first-pass static adjudication of 82 findings. This adjudication verifies static codebase evidence but is not an independent audit of every CVE status and does not validate application-level exploitability. Recall over naturally occurring external vulnerabilities is not claimed because no complete independent ground truth exists for the external codebases.

The injected external campaign provides complete ground truth only for the injected findings, not for every vulnerability

TABLE XI
ARTIFACT COMPONENTS, UTILITY, AND EXECUTION REQUIREMENTS.

Artifact component	Utility for reviewers	Main requirements
Study scripts under `SecFlowOps/artifact`	Regenerate controlled, full-protocol, external, NodeGoat, injected-external, ZAP, and npm-baseline campaigns.	Windows PowerShell, Python 3.13, host tools declared in `tool_status.json`.
Raw and processed data under `SecFlowOps/data`	Inspect scanner outputs, normalized findings, run metrics, remediation metrics, and timing records.	No execution required for inspection; reruns require scanner availability and current advisory databases.
Tables and figures under `SecFlowOps/tables` and `SecFlowOps/figures`	Audit the manuscript tables and result figures from processed metrics.	Python analysis scripts and plotting dependencies from artifact requirements.
Policies under `SecFlowOps/policies`	Verify the OPA gate used for C3 and C5 decisions.	OPA 1.18.2 or compatible Rego runtime.
Bundled tools under `SecFlowOps/tools`	Provide local OPA, Gitleaks, and ZAP binaries used in the reported environment.	Java 24 for ZAP; Windows-compatible execution for bundled binaries.
GitHub Actions evidence	Check remote CI, pull-request creation, owner merge acceptance, merge latency, and post-merge CI.	GitHub access to the repository evidence; no local rerun required.
Checksums and tool status	Verify artifact integrity and tool versions used for the reported run.	SHA-256 verification utility; awareness that scanner advisory databases evolve.

that may naturally exist in the base codebases. External campaigns use an explicit build-skip mode: they measure security-pipeline behavior, but they do not prove that remediated third-party applications still pass their full project-specific test suites. The NodeGoat npm remediation uses non-breaking `npm audit fix`; remaining findings require breaking upgrades, replacement of unmaintained packages, or manual redesign.

The toolchain is time-sensitive because scanner databases, npm advisory metadata, and package-manager resolution behavior change. The reported run fixes tool versions and integrity metadata, but future runs may produce different vulnerability counts as advisory databases evolve.

The controlled precision estimate treats scanner findings not linked to injected ground truth as unknown or false-positive candidates. The external adjudication improves this point for the reviewed findings, but it remains a static first-pass review rather than a blinded multi-reviewer audit. Stronger claims about external false-positive rates require independent reviewers and a broader corpus.

Remote CI execution and real pull-request creation were performed for the reported validation. The remote validation covers compilation, unit tests, OPA policy tests, workflow syntax validation, automated remediation-branch creation, user-pushed remediation review flow, passing pull-request CI, owner merge acceptance, measured merge latency, and passing post-merge CI. This remains a functional validation rather than a measurement of independent human review quality, branch protection policy, multi-reviewer governance, or production deployment.

IX. CONCLUSION

SecFlowOps executes real scanner, remediation, policy, metric, adjudication, DAST, and performance workflows. The full controlled protocol executes 432/432 planned local runs across three codebase families, eight scenarios, six configurations, and three repetitions. The controlled size study, the 75-run open-source campaign, the NodeGoat npm-remediation study, and the injected external recall study provide complementary checks on size effects, external codebases, package-manager remediation, and injected ground-truth matching. The ZAP probe validates executable OWASP ZAP scanning, JSON normalization, and reflected-XSS ground-truth matching under the reported Java 24 environment.

The resulting contribution is an evaluable DevSecOps framework and measurement protocol, not a production-ready autonomous remediation system. The evidence supports claims about scanner execution, controlled and injected recall, partial external remediation, policy gating, ZAP-enabled DAST execution, measured overhead, and remote CI-backed remediation flow. It does not support complete recall over naturally occurring OSS vulnerabilities, exploitability of every scanner finding, compatibility preservation for all remediated applications, independent human review quality, or production deployment.

REFERENCES

[1] B. Johnson, Y. Song, E. Murphy-Hill, and R. Bowdidge, "Why don't software developers use static analysis tools to find bugs?" in *2013 35th International Conference on Software Engineering (ICSE)*. IEEE, 2013, pp. 672–681. [Online]. Available: https://doi.org/10.1109/ICSE.2013.6606613

[2] M. Beller, R. Bholanath, S. McIntosh, and A. Zaidman, "Analyzing the state of static analysis: A large-scale evaluation in open source software," in *2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*. IEEE, 2016, pp. 470–481. [Online]. Available: https://doi.org/10.1109/SANER.2016.105

[3] R. M. Parizi, K. Qian, H. Shahriar, F. Wu, and L. Tao, "Benchmark requirements for assessing software security vulnerability testing tools," in *2018 IEEE 42nd Annual Computer Software and Applications Conference (COMPSAC)*. IEEE, 2018. [Online]. Available: https://doi.org/10.1109/COMPSAC.2018.00139

[4] A. Decan, T. Mens, and E. Constantinou, "On the impact of security vulnerabilities in the npm package dependency network," in *Proceedings of the 15th International Conference on Mining Software Repositories*. ACM, 2018, pp. 181–191. [Online]. Available: https://doi.org/10.1145/3196398.3196401

[5] S. Mirhosseini and C. Parnin, "Can automated pull requests encourage software developers to upgrade out-of-date dependencies?" in *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2017, pp. 84–94. [Online]. Available: https://doi.org/10.1109/ASE.2017.8115621

[6] National Institute of Standards and Technology, "Strategies for the integration of software supply chain security in devsecops ci/cd pipelines," National Institute of Standards and Technology, Tech. Rep. NIST SP 800-204D, 2025. [Online]. Available: https://csrc.nist.gov/pubs/sp/800/204/d/final

[7] M. Souppaya, K. Scarfone, and D. Dodson, “Secure software development framework (ssdf) version 1.1: Recommendations for mitigating the risk of software vulnerabilities,” National Institute of Standards and Technology, Tech. Rep. NIST SP 800-218, 2022. [Online]. Available: https://doi.org/10.6028/NIST.SP.800-218
[8] OWASP Foundation, “OWASP SAMM: Software assurance maturity model,” 2026, accessed 13 July 2026. [Online]. Available: https://owasp.org/www-project-samm/
[9] H. Myrbakken and R. Colomo-Palacios, “Devsecops: A multivocal literature review,” in *Communications in Computer and Information Science*. Springer International Publishing, 2017, pp. 17–29. [Online]. Available: https://doi.org/10.1007/978-3-319-67383-7_2
[10] Aqua Security, “Trivy documentation,” 2026, accessed 13 July 2026. [Online]. Available: https://trivy.dev/docs/latest/guide/
[11] Semgrep, “Semgrep documentation,” 2026, accessed 13 July 2026. [Online]. Available: https://docs.semgrep.dev/
[12] Gitleaks Project, “Gitleaks,” 2026, accessed 13 July 2026. [Online]. Available: https://github.com/gitleaks/gitleaks
[13] npm, Inc., “npm-audit cli documentation,” 2026, accessed 13 July 2026. [Online]. Available: https://docs.npmjs.com/cli/v11/commands/npm-audit/
[14] OWASP ZAP, “Zap documentation,” 2026, accessed 13 July 2026. [Online]. Available: https://www.zaproxy.org/docs/
[15] Open Policy Agent, “Open policy agent documentation,” 2026, accessed 13 July 2026. [Online]. Available: https://www.openpolicyagent.org/docs